ENGINEERING PROJECT REPORT

Automated Ball Lifting, Sorting & Shooting System

with Score Display & Bluetooth GUI Control

Name : Zeyad ahmed abdelhamid Id: 24p0440

Name : dawood Mohamed id: 24p0143

1. Project Overview

This project is an automated ball management system that performs three sequential operations: lifting a ball using a servo motor, shooting it via a DC motor, and sorting it by colour using a second servo motor guided by an IR sensor. A 16x2 I2C LCD display shows the current ball colour and the accumulated score in real time. The entire system is controlled by two Arduino microcontrollers communicating through a digital handshake, and can also be operated wirelessly via a Bluetooth module connected to a custom MIT App Inventor application.

2. System Architecture

2.1 Block Diagram

The diagram below illustrates the high-level relationship between all major subsystems:

2.2 Arduino Communication

Arduino 1 acts as the master controller. Once it completes the lifting and shooting cycle, it sends a HIGH signal on Pin 12 to Arduino 2 (slave) via Pin 12. The shared ground and VCC lines ensure a common reference voltage. This simple one-wire handshake replaces I²C or SPI for the inter-Arduino link, keeping wiring minimal.

3. Hardware Components

#	Component	Role in System
1	**Arduino Uno (x2)**	Microcontrollers – Master (Arduino 1) and Slave (Arduino 2)
2	**Servo Motor (Lifting)**	Rotates the lifting arm from resting (10°) to raised (130°) position
3	**Servo Motor (Sorting)**	Deflects the ball to the correct channel: 0° for black, 160° for white
4	**DC Motor + H-Bridge (L298N)**	Shoots the ball by spinning at full speed for 4 seconds

5	**IR Sensor (x2 channels)**	Detects ball presence and reflects light to distinguish black vs. white
6	**16x2 I2C LCD Display**	Shows ball colour (BLACK / WHITE) and cumulative score
7	**HC-06 Bluetooth Module**	Wirelessly receives Up, Down, and Start commands from the mobile app
8	**Push Button**	Manual trigger for starting the automated cycle on Arduino 1
9	**Power Supply (5 V / bench)**	Provides regulated power to the entire circuit
10	**3D-Printed Frame**	Red/Blue PLA structure: lifting arm, shooting ramp, sorting gate, and funnel

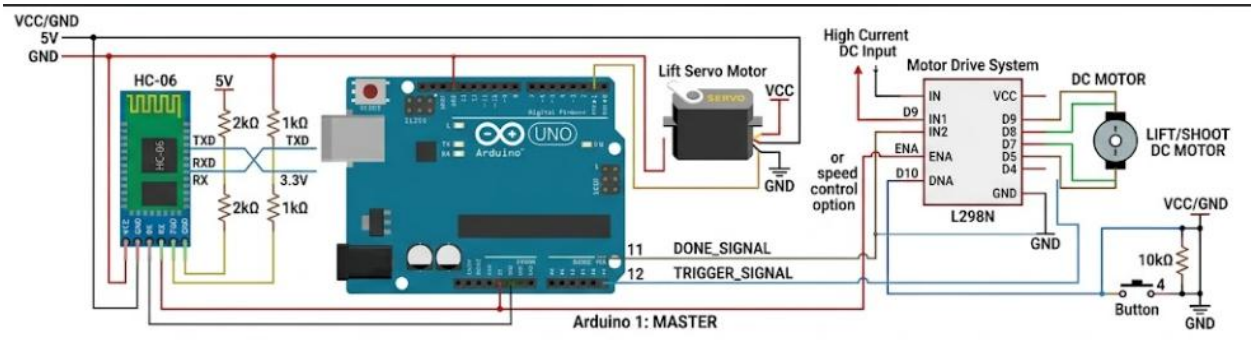
4. Circuit Diagrams & Schematics

4.1 Arduino 1 – Lifting & Shooting Circuit

The table below describes every pin connection on Arduino 1. The Fritzing / schematic diagram placeholder follows.

Arduino 1 Pin	Connected To	Description
D6 (PWM)	Lift Servo signal	PWM signal controls lifting servo position (10° rest / 130° raised)
D9	L298N IN1	Motor direction control pin 1 for DC shooter motor
D8	L298N IN2	Motor direction control pin 2 for DC shooter motor
D10 (PWM)	L298N ENA	Speed control (PWM) for DC motor – wired but currently driven HIGH/LOW
D12	Arduino 2 Pin 12	Trigger / handshake line: HIGH = Arduino 2 should start sorting
D11	Arduino 2 Pin 11	DONE signal from Arduino 2 (input, pull-up)
D4	Push button	Physical start button (INPUT_PULLUP, active LOW)
TX / RX	HC-06 RX / TX	UART serial Bluetooth communication at 9600 baud
5V / GND	HC-06 VCC / GND	Power for Bluetooth module
5V / GND	L298N logic power	Logic-level power for H-bridge

Circuit diagram – Arduino 1:

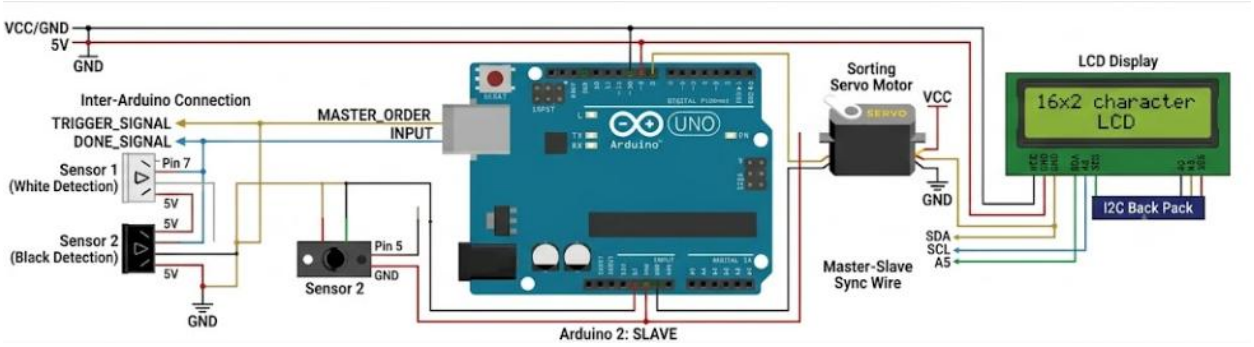


4.2 Arduino 2 – Sorting & Display Circuit

Arduino 2 Pin	Connected To	Description
D9 (PWM)	Sort Servo signal	PWM signal controls sorting servo (0° black / 70° neutral / 160° white)

D7	IR sensor OUT (white)	Digital read: 0 = white ball detected, 1 = no white reflected
D5	IR sensor OUT (black)	Digital read: 1 = black ball detected, 0 = no black reflected
D12	Arduino 1 Pin 12	Receives HIGH trigger signal from master to begin sorting
SDA (A4)	LCD SDA	I2C data line for 16x2 hd44780 LCD
SCL (A5)	LCD SCL	I2C clock line for 16x2 hd44780 LCD
5V / GND	LCD VCC / GND	Power for LCD backlight and logic
5V / GND	IR sensors VCC / GND	Power for both IR sensor modules

Circuit diagram – Arduino 2:



5. Operational Flow

5.1 Full System Flowchart

The flowchart below captures the complete cycle from power-on to score update:

START / POWER ON

↓

Arduino 1 initialises: servo to 10°, motor stopped, Bluetooth listening

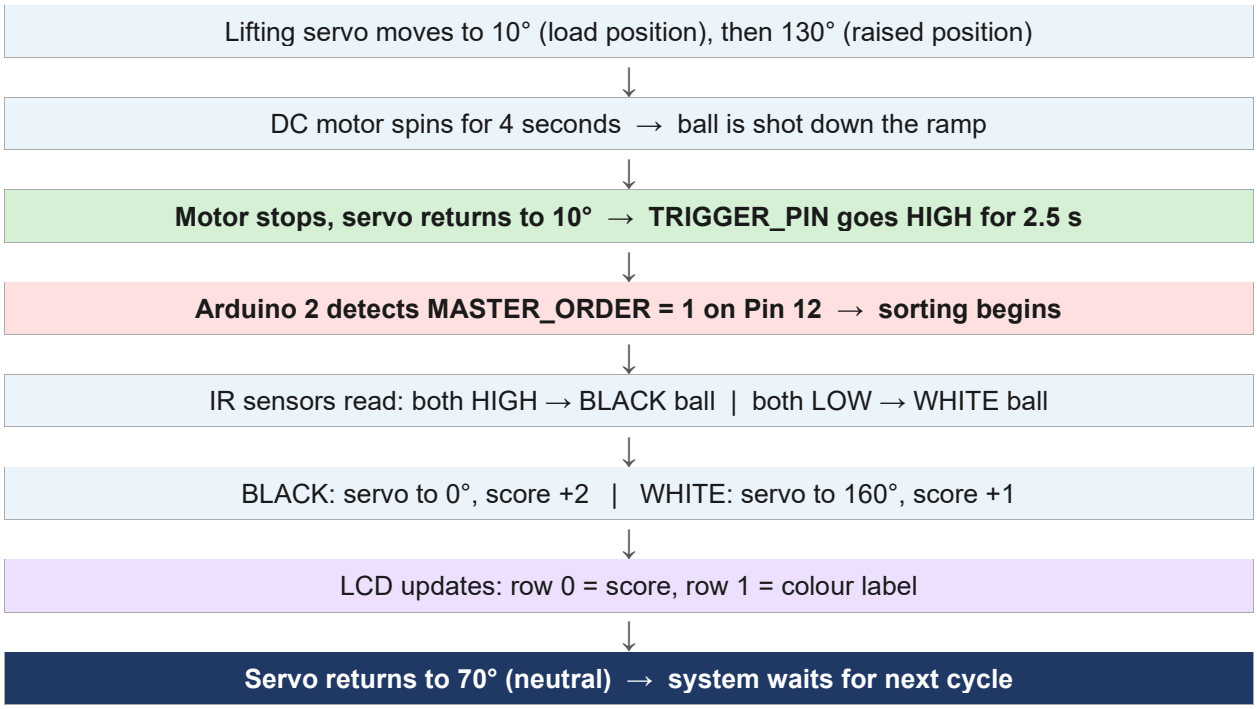
↓

[Optional] MIT App sends U / D to manually position lifting arm

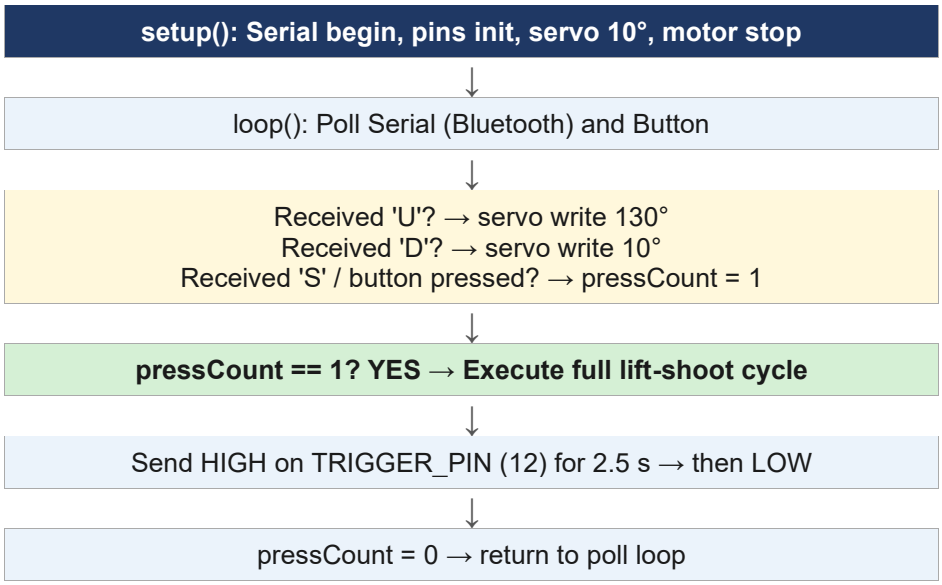
↓

App sends 'S' OR physical button pressed → pressCount = 1

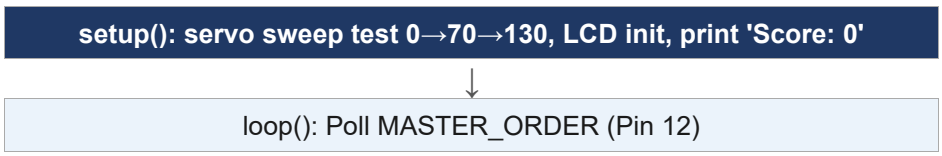
↓

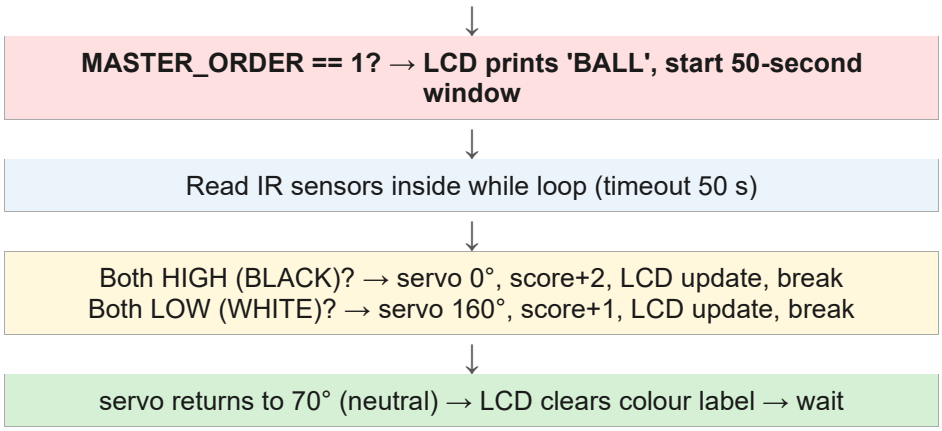


5.2 Arduino 1 Flowchart (Lifting & Shooting)



5.3 Arduino 2 Flowchart (Sorting & Display)




MASTER_ORDER == 1? → LCD prints 'BALL', start 50-second window
Read IR sensors inside while loop (timeout 50 s)
Both HIGH (BLACK)? → servo 0°, score+2, LCD update, break
Both LOW (WHITE)? → servo 160°, score+1, LCD update, break
servo returns to 70° (neutral) → LCD clears colour label → wait

6. Subsystem Descriptions

6.1 Lifting Subsystem

The lifting arm is a 3D-printed red ramp mounted on a blue pivot block. A servo motor (connected to Arduino 1 Pin D6) rotates between two positions: 10° is the loading position where the arm waits for a ball to roll into the cup at its base, and 130° is the raised position that tilts the arm, allowing the ball to roll toward the shooting mechanism. The 1.5-second delays after each movement ensure the mechanical structure settles before the next action.

6.2 Shooting Subsystem

A DC motor driven through an L298N H-bridge provides the shooting force. Arduino 1 sets IN1 HIGH and IN2 LOW to spin the motor in one direction. After a 4-second spin, the M_STOP() function sets both control pins LOW, cutting power. The ENA pin is currently not PWM-modulated in the final code, meaning the motor runs at full supply voltage during the shot. The red ramp guides the ball toward the target area.

6.3 Sorting Subsystem

Once the trigger signal from Arduino 1 is received on Pin 12, Arduino 2 reads two IR sensor outputs. The sensors are tuned so that a black ball produces both outputs HIGH (due to low infrared reflectance), while a white ball produces both outputs LOW (high reflectance). A sorting servo (Pin D9 on Arduino 2) deflects to 0° for black balls and 160° for white balls, physically directing each ball into its respective bin. After a 3-second settling delay, the servo returns to 70° (neutral, centred between the two exit channels).

6.4 Score Display

The 16x2 I2C LCD module uses the hd44780 library to display two rows of information simultaneously. Row 0 (top) shows the running total score in the format 'Score: X'. Row 1 (bottom, starting at column 5) shows the most recently detected colour: 'BALL' while waiting, 'BLACK' or 'White' once identified, and then blanked until the next ball arrives. The scoring rules are: black ball = 2 points, white ball = 1 point.

6.5 Bluetooth GUI Control

An HC-06 Bluetooth module is connected to the hardware UART of Arduino 1 (TX/RX pins). It communicates at 9600 baud. The MIT App Inventor application pairs with the HC-06 and provides three buttons:

- Up (sends character 'U') – moves the lifting servo to 130° for manual positioning.
- Down (sends character 'D') – returns the lifting servo to 10°.
- Start (sends character 'S') – sets pressCount to 1, triggering the full automated cycle.

This allows the operator to position the arm before beginning, ensuring a ball is properly loaded, and then start the closed-loop cycle from the app without needing physical access to the button.

7. Source Code

7.1 Arduino 1 – Lifting, Shooting & Bluetooth

This code handles the lifting servo, DC motor shooter, push button input, and Bluetooth command parsing. It signals Arduino 2 via TRIGGER_PIN (Pin 12) when a cycle completes.

```
#include <Servo.h>
#define IN1 9
#define IN2 8
#define ENA 10
#define SERVO_PIN 6
#define TRIGGER_PIN 12
#define BUTTON_PIN 4
#define DONE_PIN 11

Servo liftServo;
int pressCount = 0;
bool lastButtonState = HIGH;

void setup() {
  delay(2000);
  Serial.begin(9600);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(TRIGGER_PIN, OUTPUT);
  digitalWrite(TRIGGER_PIN, HIGH);
  liftServo.attach(SERVO_PIN);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  liftServo.write(10);
  delay(1000);
  M_STOP();
  liftServo.write(10);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(DONE_PIN, INPUT_PULLUP);
  lastButtonState = digitalRead(BUTTON_PIN);
  Serial.println("System Ready. Waiting for App or Button command...");
}

void loop() {
  if (Serial.available() > 0) {
    char guiCommand = Serial.read();
    if (guiCommand == 'U') {
      liftServo.write(130); // Manual Up
    } else if (guiCommand == 'D') {
      liftServo.write(10);  // Manual Down
    } else if (guiCommand == 'S') {
      pressCount = 1;       // Start cycle
    }
  }
  checkButton();
  if (pressCount == 1) {
    liftServo.write(10);
    delay(1500);
    liftServo.write(130);
    delay(1500);
```

```
    delay(5000);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);
    delay(4000);
    M_STOP();
    liftServo.write(10);
    Serial.println("Lifting and shooting done. Signaling Arduino 2...");
    pressCount = 0;
    digitalWrite(TRIGGER_PIN, HIGH);
    delay(2500);
    digitalWrite(TRIGGER_PIN, LOW);
  }
}

void checkButton() {
  bool currentButtonState = digitalRead(BUTTON_PIN);
  if (lastButtonState == HIGH && currentButtonState == LOW) {
    delay(50);
    if (digitalRead(BUTTON_PIN) == LOW) {
      pressCount++;
    }
  }
  lastButtonState = currentButtonState;
}

void M_STOP() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
}
```

7.2 Arduino 2 – Sorting, IR Sensing & LCD Display

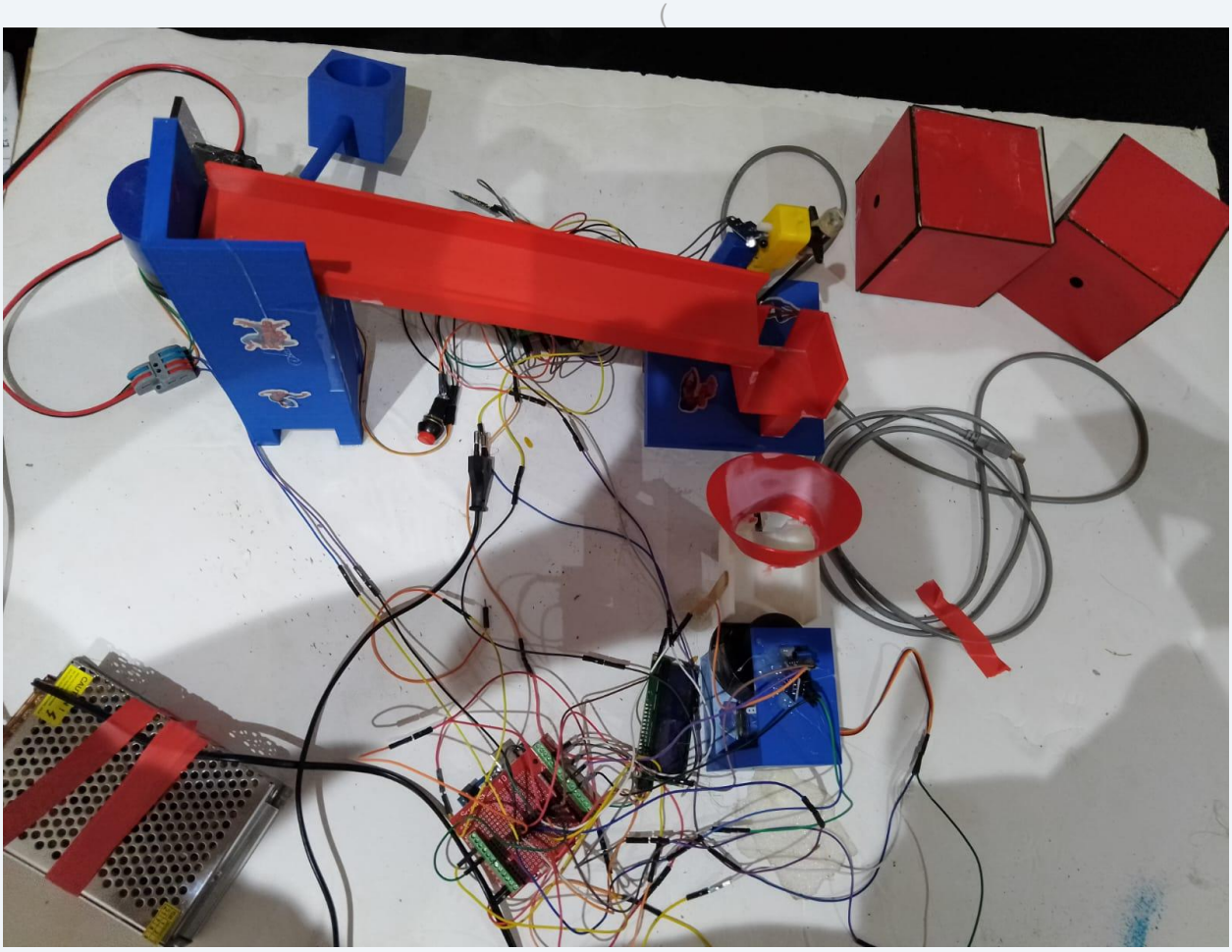
This code waits for the handshake from Arduino 1, reads both IR sensors to identify ball colour, actuates the sorting servo, updates the score, and prints everything to the LCD.

```
#include <Wire.h>
#include <hd44780.h>
#include <hd44780ioClass/hd44780_I2Cexp.h>
#include <Servo.h>

#define WHITE_read digitalRead(7)
#define BLACK_READ digitalRead(5)
#define MASTER_ORDER digitalRead(12)

hd44780_I2Cexp lcd;
Servo myServo;
int redValue, blueValue;
int score = 0;

void setup() {
  pinMode(7, INPUT);
  pinMode(5, INPUT);
  myServo.attach(9);
  delay(500);
  myServo.write(0);
  delay(500);
  myServo.write(70);
```

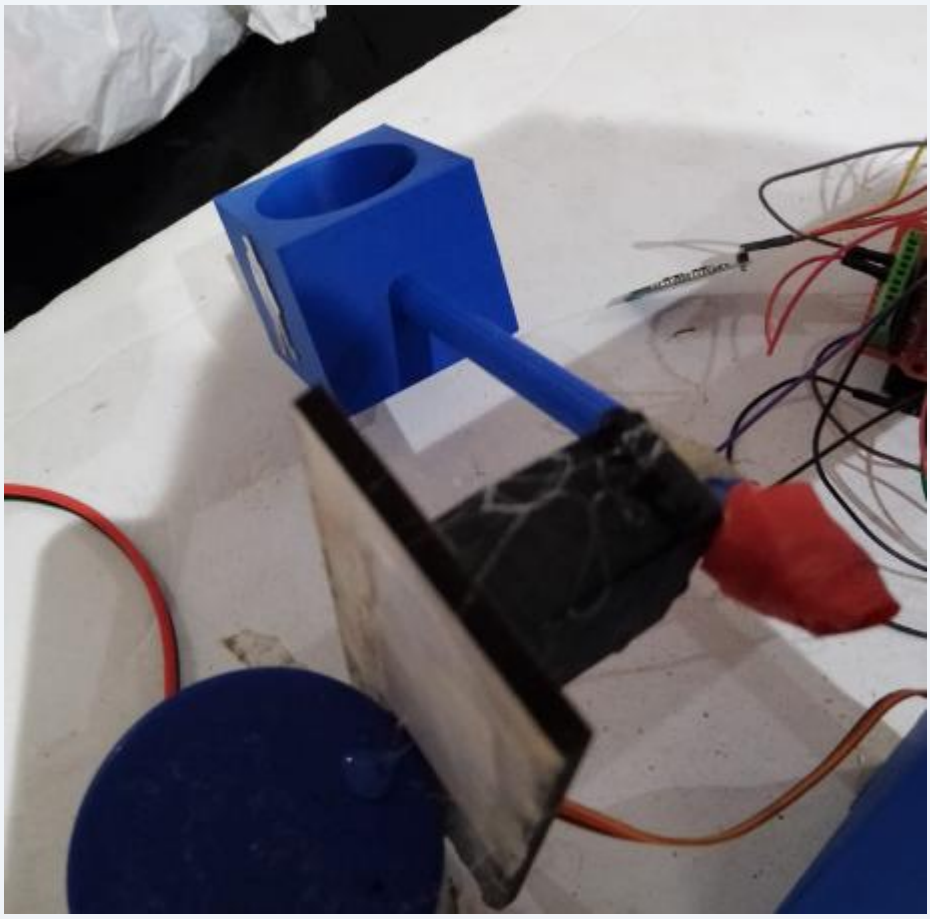
```
  delay(500);
  myServo.write(130);
  delay(500);
  Serial.begin(9600);
  lcd.begin(16, 2);
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Score: 0");
}

void loop() {
  unsigned long startTime = millis();
  if (MASTER_ORDER == 1) {
    Serial.println("HAVE BALL ");
    lcd.setCursor(1, 5);
    lcd.print("BALL");
    delay(1000);
    while (millis() - startTime < 50000) {
      if (BLACK_READ == 1 && WHITE_read == 1) {
        lcd.setCursor(1, 5);
        lcd.print("BLACK");
        Serial.println("  BLACK: ");
        myServo.write(0);
        delay(3000);
        score += 2;
        lcd.setCursor(0, 0);
        lcd.print("Score: ");
        lcd.print(score);
        lcd.print("   ");
        break;
      } else if (BLACK_READ == 0 && WHITE_read == 0) {
        Serial.println("  White: ");
        lcd.setCursor(1, 5);
        lcd.print("White ");
        myServo.write(160);
        delay(3000);
        score += 1;
        lcd.setCursor(0, 0);
        lcd.print("Score: ");
        lcd.print(score);
        lcd.print("   ");
        break;
      }
      myServo.write(70);
    }
  }
  myServo.write(70);
  lcd.setCursor(1, 5);
  lcd.print("    ");
}
```

8. Project Photos

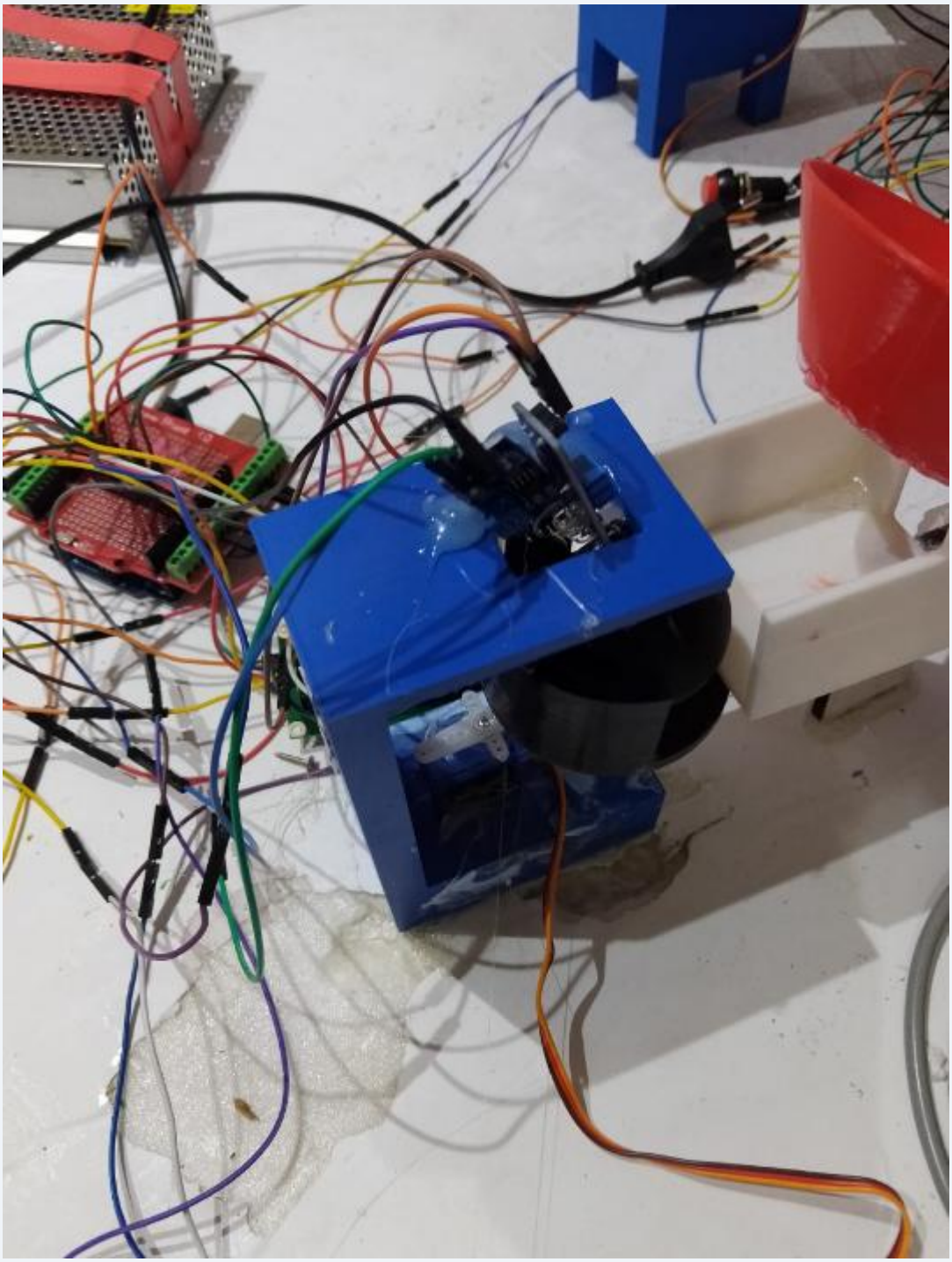
Full Project Overview



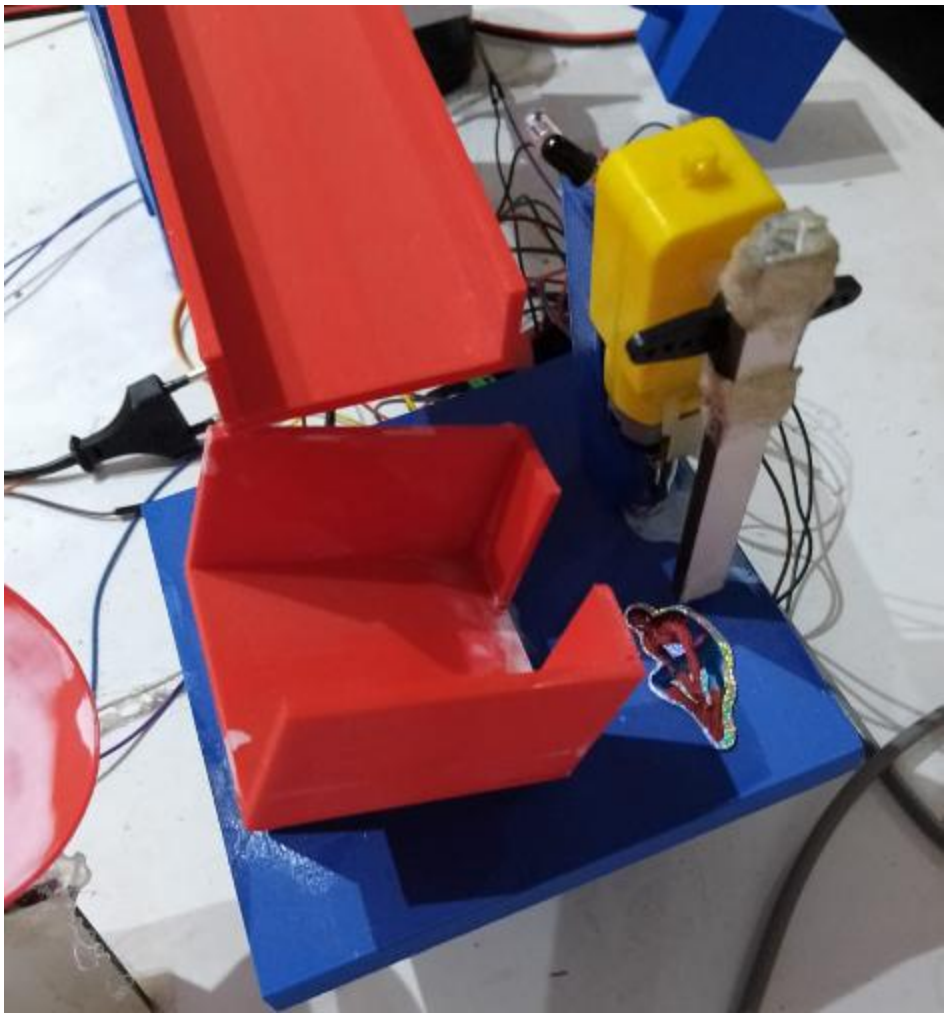
8.2 Lifting Arm & Servo



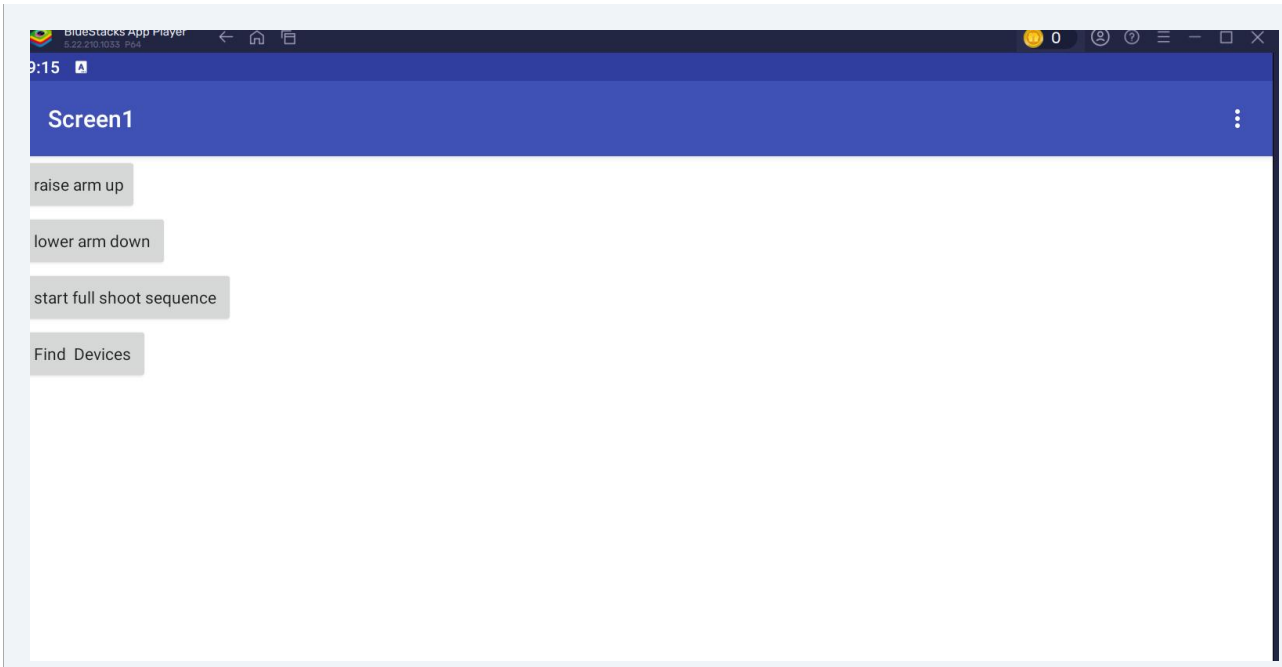
8.3 Sorting Gate & IR Sensors

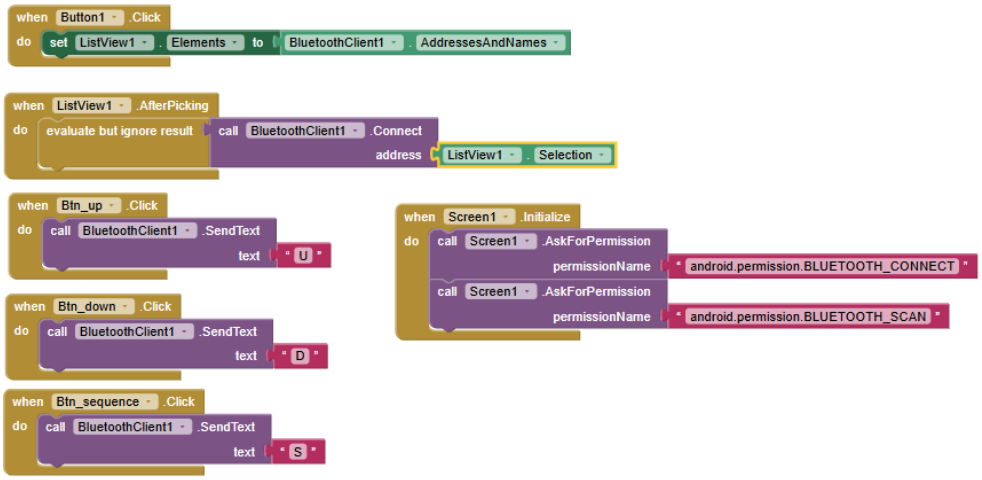


shooting and dc motor



8.6 MIT App Inventor GUI Screenshot


BlueStacks App Player
5.22.210.1033 P64
0
9:15
Screen1
raise arm up
lower arm down
start full shoot sequence
Find Devices



9. Conclusion

This project successfully demonstrates a fully automated, multi-stage mechatronic system capable of lifting, shooting, and sorting balls while tracking and displaying a score. The division of responsibilities between two Arduinos—connected by a simple digital handshake—kept each subsystem's code clean and manageable. Bluetooth integration via the HC-06 module and a custom MIT App Inventor application added a user-friendly wireless interface, allowing the lifting arm to be manually positioned before each cycle begins.

The IR-based colour detection reliably distinguishes black balls (low reflectance, both sensors HIGH) from white balls (high reflectance, both sensors LOW). The LCD provides clear real-time feedback to the user, reinforcing the competitive scoring element of the system.

Potential future improvements include adding a third ball colour using an RGB colour sensor, implementing bidirectional communication between the two Arduinos for more robust synchronisation, and integrating a score-reset button accessible through the mobile application.

Appendix – Project Video

Scan the QR code below to watch the project demonstration video: